МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВАСТОПОЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
ВТШ «СПИ»

ОТЧЕТ

о выполнении лабораторной работы № 2

Тема: «МЕТОДЫ ПОИСКА МИНИМУМА ФУНКЦИИ ОТ ОДНОЙ ПЕРЕМЕННОЙ, ОСНОВАННЫЕ НА ПОЛИНОМИАЛЬНОЙ АППРОКСИМАЦИИ»

по дисциплине

«ПРИКЛАДНАЯ МАТЕМАТИКА В ИИ»

Вариант № 4

Выполнил:
Студент группы ИИ/б-25-6-о
Заварзин А. В.
Проверила:
к.т.н., доцент, доцент кафедры ПМиИ,
доцент кафедры ИТиС
Ченгарь О. В.

Севастополь, 2026

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. В MS Excel построить таблицу значений заданной функции и ее график на отрезке, выбранном в лабораторной работе 1.

2. Определить локальный минимум функции f(x) на основе методов полиномиальной аппроксимации, выполнив 3 итерации (с помощью MS Excel). Начальной координатой поиска считать один из концов интервала унимодальности [a, b], определенного для функций f(x) в первой лабораторной работе:

а) методом квадратической аппроксимации;

б) методом кубической аппроксимации

3. Решить задачу всеми методами, используя Python.

4. Провести сравнительный анализ результатов поиска экстремума методами полиномиальной аппроксимации, а также сравнить полученные результаты с результатами поиска экстремума функции методами исключения интервалов.

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ ПО ВАРИАНТУ

Для варианта № 4 дана следующая функция:

$$f(x) = x^4 + 8x^3 - 6x^2 - 72x + 90$$

Для вычислений также возьмём значение погрешностей:

$$\varepsilon_1 = 0{,}003; \quad \varepsilon_2 = 0{,}003$$

ХОД РАБОТЫ

Задание 0

Из данной для варианта №4 функции найдём точки экстремума.

Для начала найдём производную функции:

$$f'(x) = 4x^3 + 24x^2 - 12x - 72$$

Далее приравниваем производную к нулю:

$$4x^3 + 24x^2 - 12x - 72 = 0$$

Находим корни:

$$\begin{cases} x_1 = -6 \\ x_2 = -\sqrt{3} \\ x_3 = \sqrt{3} \end{cases}$$

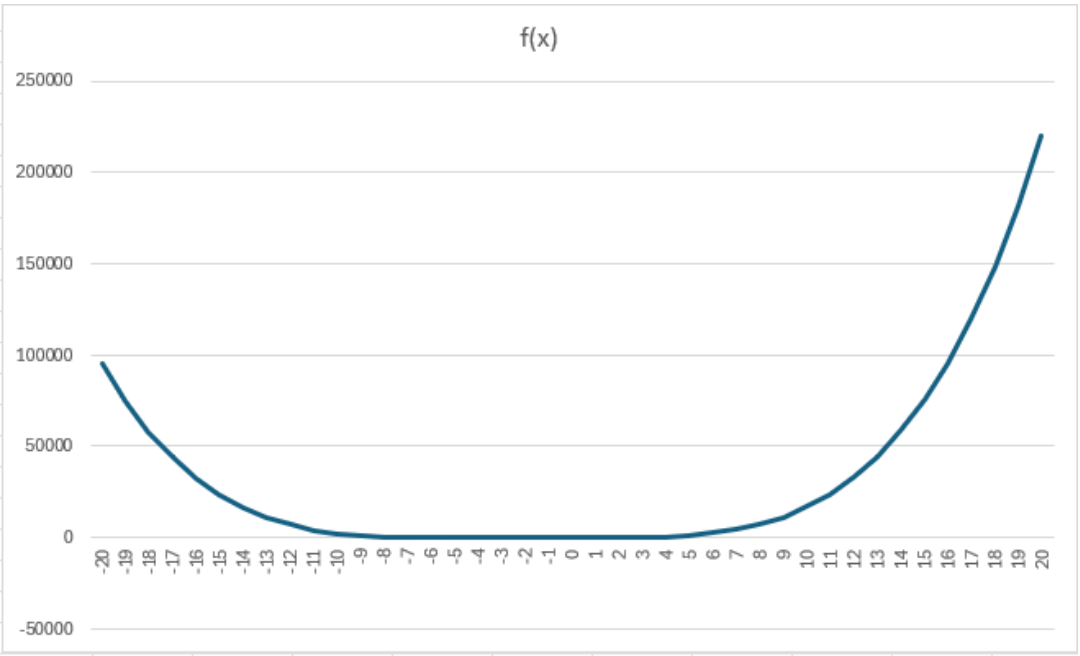
Подставляя все полученные значения в исходную функцию можно сказать, что точка минимума функции по оси располагается в -6. А значение функции в точке минимума равно -126.

Задание 1.

В начале необходимо построить таблицу значений функции в MS Excel, а затем, по полученной таблице, построить её график. Исходя из анализа графика в лабораторной работе 1 был выбран аналогичный интервал [-10; 6].

Результат построения таблицы значения и графика функции в среде MS Excel представлен на рисунке 3.1.

x	f(x)
-20	95130
-19	74741
-18	57762
-17	43797
-16	32474
-15	23445
-14	16386
-13	10997
-12	7002
-11	4149
-10	2210
-9	981
-8	282
-7	-43
-6	-126
-5	-75
-4	26
-3	117
-2	162
-1	149
0	90
1	21
2	2
3	117
4	474
5	1205
6	2466
7	4437
8	7322
9	11349
10	16770
11	23861
12	32922
13	44277
14	58274
15	75285
16	95706
17	119957
18	148482
19	181749
20	220250



a	-10
b	6
ε_1	0,0003
ε_2	0,003
Δx	1

Рисунок 3.1 – Построенная таблица значений и график для функции

Задание 2

После построения графика, выбора значений некоторых констант и получения интервала для поиска минимума был произведён поиск точки минимума заданной функции с помощью методов квадратичной и кубической аппроксимации. В среде MS Excel были произведены первые 3 итерации для каждого метода. Результаты вычислений методом квадратичной и кубической аппроксимации представлены на рисунках 3.2 и 3.3 соответственно.

Итерация	x1	x2	x3	f(x1)	f(x2)	f(x3)	x_min	f_min	x̄	f(x̄)	a1	a2	Условие 1	Условие 2
1	-10	-9	-8	2210	981	282	-8	282	-7,18113	-5,61471	-1229	265	0	0
2	-7,18113	-6,18113	-5,18113	-5,61471	-123,738	-90,0798	-6,18113	-123,738	-5,90289	-125,392	-118,12375	75,891193	0	0
3	-6,18113	-5,90289	-5,18113	-123,738	-125,392	-90,0798	-5,90289	-125,392	-5,98785	-125,99	-5,9432622	54,868826	0	0

Рисунок 3.2 – Результат первых трёх итераций квадратичной аппроксимации

Итерация	x1	x2	f(x1)	f(x2)	df(x1)	df(x2)	x̄	f(x̄)	df(x̄)	z	μ	ω	Условие 1	Условие 2
1	-7	-3	-43	117	-184	72	-5,9096	-125,472	11,54329862	-232	0,7274	258,983	0	0
2	-7	-5,9096	-43	-125,472	-184	11,5433	-6,00126	-126	-0,16607069	54,4486	0,08406	71,3346	0	0
3	-6,00126	-5,9096	-126	-125,472	-0,16607	11,5433	-6	-126	2,02693E-05	-5,88799	0,98628	6,04859	1	1

Рисунок 3.3 – Результат первых трёх итераций кубической аппроксимации

Задание 3

После изучения алгоритмов всех методов и произведения вычислений в MS Excel необходимо написать код на языке Python, который будет находить точку минимума функции с заданной точностью по всем двум методам аппроксимации, а также для удобства, можно реализовать построение графиков с показанной на них функцией и отмеченной точкой минимума, а также возможность изменения констант (границы интервала, погрешности, размер шага). Для построения графиков будет использована библиотека «matplotlib», также для вывода таблиц с параметрами на каждой итерации будет использована библиотека «Rich», а для числовых вычислений (например, генерация равномерно распределённого интервала) будет использована библиотека «NumPy». Листинг кода представлен в приложении А.

Результаты работы программы представлены на рисунках 3.4 – 3.7.

```
======== МЕНЮ ========
1. Метод квадратичной аппроксимации
2. Метод кубической аппроксимации
3. Показать график функции на отрезке [-10; 6]
4. Изменить настройки программы
5. Выход
-> 1
```

Квадратичная аппроксимация

Итерации	x1	x2	x3	f(x1)	f(x2)	f(x3)	x_min	f_min	x_hat	f(x_hat)	a1	a2	Условие 1	Условие 2
1	-10	-9	-8	2210	981	282	-8	282	-7.181132	-5.614708	-1229.000000	265.000000	✗	✗
2	-7.181132	-6.181132	-5.181132	-5.614708	-123.738457	-90.079822	-6.181132	-123.738457	-5.902888	-125.392135	-118.123750	75.891193	✗	✗
3	-6.181132	-5.902888	-5.181132	-123.738457	-125.392135	-90.079822	-5.902888	-125.392135	-5.987851	-125.990287	-5.943262	54.868826	✗	✗
4	-6.181132	-5.987851	-5.902888	-123.738457	-125.990287	-125.392135	-5.987851	-125.990287	-5.997772	-125.999673	-11.650559	67.173716	✓	✓

```
Результат вычисления по методу КВАДРАТИЧНОЙ АППРОКСИМАЦИИ:
• Точка минимума: x_min = -5.997772 ± 0.0003
• Значение в точке минимума: f(x_min) = -125.999673 ± 0.003
• Заняло итераций: 4
```

Рисунок 3.4 – Результат вычисления по методу квадратичной аппроксимации с помощью программы на языке Python



Рисунок 3.5 – График функции с найденной точкой минимума для метода квадратичной аппроксимации

```
======== МЕНЮ ========
1. Метод квадратичной аппроксимации
2. Метод кубической аппроксимации
3. Показать график функции на отрезке [-10; 6]
4. Изменить настройки программы
5. Выход
-> 2
```

Кубическая аппроксимация

Итерации	x1	x2	f(x1)	f(x2)	df(x1)	df(x2)	z	w	mu	x_hat	f(x_hat)	df(x_hat)	Условие 1	Условие 2
1	-7	-3	-43	117	-184	72	-232.000000	258.982625	0.727400	-5.909602	-125.472409	11.543299	✗	✗
2	-7	-5.909602	-43	-125.472409	-184	11.543299	54.448637	71.334571	0.084057	-6.001258	-125.999896	-0.166071	✗	✗
3	-6.001258	-5.909602	-125.999896	-125.472409	-0.166071	11.543299	-5.887987	6.048586	0.986278	-6.000000	-126.000000	0.000020	✓	✓

```
Результат вычисления по методу КУБИЧЕСКОЙ АППРОКСИМАЦИИ:
• Точка минимума: x_min = -6.000000 ± 0.0003
• Значение в точке минимума: f(x_min) = -126.000000 ± 0.003
• Заняло итераций: 3
=========================================================
```

Рисунок 3.6 – Результат вычисления по методу кубической аппроксимации с помощью программы на языке Python

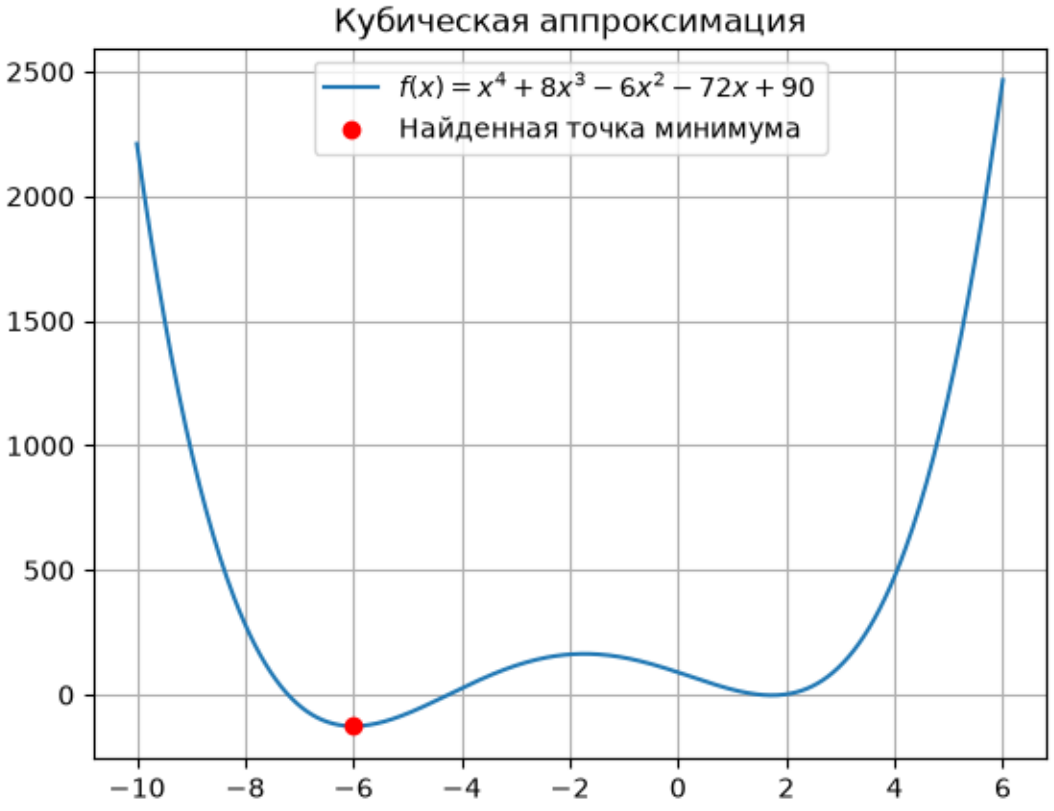


Рисунок 3.7 – График функции с найденной точкой минимума для метода кубической аппроксимации

Задание 4

После вычисления точки минимума разными методами с помощью разных инструментов (инструментами MS Excel и с помощью языка Python) можно сделать сравнение результатов вычислений.

Сравнение методов полиномиальной аппроксимации показало, что оба метода успешно находят точку минимума $x^* = -6,\ f(x^*) = -126$, однако кубическая аппроксимация оказалась эффективнее. Метод квадратичной аппроксимации потребовал 4 итерации и дал результат $x \approx -5{,}99772$ (абсолютная погрешность около $2{,}3 \cdot 10^{-3}$), тогда как кубический метод справился за 3 итерации, попав практически точно в $x = -6{,}000000$. Это объясняется тем, что кубический полином использует значения не только самой функции, но и её производных на границах интервала, что позволяет точнее описать кривизну функции и быстрее сойтись к экстремуму. Квадратичная аппроксимация, опирающаяся только на значения функции, уступает в точности, но выигрывает в тех случаях, когда вычисление производных затруднено.

В сравнении с методами исключения интервалов (деления пополам, Фибоначчи и золотого сечения), рассмотренными в первой лабораторной работе, методы полиномиальной аппроксимации демонстрируют значительно более высокую скорость сходимости и точность. При заданной точности $\varepsilon = 0{,}05$ методам исключения интервалов требуется от 9 до 14 итераций, а координата минимума определяется лишь с точностью до $\pm 0{,}025$. Полиномиальные методы при аналогичных вычислительных затратах (3–4 итерации) обеспечивают точность на один–два порядка выше за счёт построения аппроксимирующей модели и прямого перехода в окрестность минимума.

ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения работы были приобретены практические навыки вычисления точки минимума методами полиномиальной аппроксимации (с помощью методов квадратичной и кубической аппроксимации).

Также приобретены навыки вычисления минимума функции с помощью инструментов MS Excel (построение итерационных таблиц с помощью формул) и посредством написания программы на языке Python с возможностью визуализации графиков и вывода таблиц итераций.

Все полученные значения минимума функции ($x^* = -6$, $f(x^*) = -126$) совпадают с теоретическим расчётом с заданной погрешностью $\varepsilon_1 = 0{,}0003$ и $\varepsilon_2 = 0{,}003$. При этом метод кубической аппроксимации показал более высокую скорость сходимости (3 итерации) по сравнению с квадратичным (4 итерации).

Сравнение с методами исключения интервалов (из лабораторной работы №1) показало, что методы полиномиальной аппроксимации позволяют достичь значительно более высокой точности за меньшее число итераций, однако требуют больших вычислений, в том числе и производных.

ПРИЛОЖЕНИЕ А

Текст программы на языке Python

```
import numpy as np
import matplotlib.pyplot as plt
import rich.console as cls
import rich.table as tbl

def f(x):
    """
    ## Заданная функция по варианту 4.
    """
    return x**4 + 8*x**3 - 6*x**2 - 72*x + 90

def df(x):
    """
    ## Производная функции по варианту 4.
    """
    return 4*x**3 + 24*x**2 - 12*x - 72

def printTable(name: str, _table: dict):
    """
    ## Вывод таблицы итераций в консоль через rich.
    """
    console = cls.Console()
    table = tbl.Table(title=name, show_lines=True,
header_style='bold')

    for col in _table.keys():
        table.add_column(header=col, justify="center")

    n_rows = max((len(v) for v in _table.values()), default=0)

    for i in range(n_rows):
        row = []
        for values in _table.values():
            if i < len(values):
                val = values[i]
                if isinstance(val, float):
                    row.append(f"{val:.6f}")
                elif isinstance(val, (bool, np.bool_)):
                    row.append("✅" if val else "❌")
                else:
                    row.append(str(val))
            else:
                row.append("—")
        table.add_row(*row)

    console.print()
    console.print(table)

def viewPlot(flag = False, start = 0, end = 0, x_min = 0, f_min =
```

```
0, title: str = ''):
    x = np.linspace(start, end, 500)
    plt.plot(x, f(x), label=r"$ f(x) = x^4 + 8x^3 - 6x^2 - 72x +
90 $")
    plt.grid(True)
    if flag:
        plt.plot(x_min, f_min, 'o', color="red", label="Найденная
точка минимума")
    plt.legend()
    plt.title(label=title)
    plt.show()

def quadraticApproximation(_x1: float, eps1: float = 0.0003, eps2:
float = 0.003, dx: int = 1):
    """
    ## Функция поиска минимума по методу квадратичной
аппроксимации.
    """
    cond1 = cond2 = skip = False
    table = {
        "Итерации": [], "x1": [], "x2": [], "x3": [],
        "f(x1)": [], "f(x2)": [], "f(x3)": [], 'x_min': [],
'f_min': [],
        "x_hat": [], "f(x_hat)": [],
        "a1": [], "a2": [],
        "Условие 1": [], "Условие 2": []
    }
    x1 = _x1
    iterations = 0
    while not (cond1 and cond2):
        iterations += 1
        if not skip:
            x2 = x1 + dx     # 2
            f_x1 = f(x1)     # 3
            f_x2 = f(x2)     # 3
            if f_x1 > f_x2:  # 4a
                x3 = x1 + 2 * dx
            else:            # 4b
                x3 = x1 - dx
            f_x3 = f(x3)     # 5

            res = sorted([(f_x1, x1), (f_x2, x2), (f_x3, x3)])

        table['Итерации'].append(iterations)
        table['x1'].append(x1)
        table['x2'].append(x2)
        table['x3'].append(x3)
        table['f(x1)'].append(f_x1)
        table['f(x2)'].append(f_x2)
        table['f(x3)'].append(f_x3)

        f_min = res[0][0]  # 6
        x_min = res[0][1]  # 6
```

```
        table['x_min'].append(x_min)
        table['f_min'].append(f_min)

        a1 = (f_x2 - f_x1) / (x2 - x1)
        a2 = (1 / (x3 - x2)) * (((f_x3 - f_x1) / (x3 - x1)) -
((f_x2 - f_x1) / (x2 - x1)))
        table['a1'].append(a1)
        table['a2'].append(a2)

        _x = 0.5 * (x2 + x1) - (a1 / (2*a2))
        f_hat = f(_x)
        table['x_hat'].append(_x)
        table['f(x_hat)'].append(f_hat)

        cond1 = np.fabs((f_min - f_hat) / f_hat) < eps1
        cond2 = np.fabs((x_min - _x) / _x) < eps2
        table['Условие 1'].append(cond1)
        table['Условие 2'].append(cond2)

        if not (cond1 and cond2):
            if min(x1, x3) <= _x <= max(x1, x3):
                pts = [(x1, f_x1), (x2, f_x2), (x3, f_x3), (_x,
f_hat)]
                pts.sort(key=lambda p: p[0])
                best_idx = min(range(4), key=lambda i: pts[i][1])

                if best_idx == 0:
                    triple = pts[:3]
                elif best_idx == 3:
                    triple = pts[-3:]
                else:
                    triple = pts[best_idx - 1: best_idx + 2]

                (x1, f_x1), (x2, f_x2), (x3, f_x3) = triple
                res = sorted([(f_x1, x1), (f_x2, x2), (f_x3, x3)])

                skip = True
            else:
                x1 = _x
                skip = False
        else:
            break

    printTable("Квадратичная аппроксимация", table)

    print(f"\n\nРезультат вычисления по методу КВАДРАТИЧНОЙ
АППРОКСИМАЦИИ:")
    print(f"• Точка минимума: x_min = {_x:.6f} ± {eps1}")
    print(f"• Значение в точке минимума: f(x_min) = {f_hat:.6f} ±
{eps2}")
    print(f"• Заняло итераций: {iterations}")
    print("="*58 + "\n\n")
```

```
    if GRAPHICS_ON:
        viewPlot(flag=True, x_min=_x, f_min=f_hat, start=A, end=B,
title='Квадратичная аппроксимация')

    return (_x, f_hat)

def cubicApproximation(_x0: float, eps1: float = 0.0003, eps2:
float = 0.003, dx: int = 1):
    """
    ## Функция поиска минимума по методу кубической аппроксимации.
    ## Алгоритм полностью соответствует методичке (шаги 1-7).
    """
    table = {
        "Итерации": [],
        "x1": [], "x2": [],
        "f(x1)": [], "f(x2)": [],
        "df(x1)": [], "df(x2)": [],
        "z": [], "w": [], "mu": [],
        "x_hat": [], "f(x_hat)": [], "df(x_hat)": [],
        "Условие 1": [], "Условие 2": []
    }

    # ============================================================
    # Шаги 1-4: поиск интервала [x1, x2], где f'(x) меняет знак
    # ============================================================
    x0 = _x0
    df_x0 = df(x0)
    direction = 1 if df_x0 < 0 else -1

    bracket = [(x0, df_x0)]
    x_prev = x0
    k = 0
    x_curr = x_prev + (2 ** k) * dx * direction
    while df(x_prev) * df(x_curr) > 0 and k < 50:
        bracket.append((x_curr, df(x_curr)))
        k += 1
        x_prev = x_curr
        x_curr = x_prev + (2 ** k) * dx * direction
    bracket.append((x_curr, df(x_curr)))

    x1, x2 = x_prev, x_curr
    if x1 > x2:
        x1, x2 = x2, x1

    f_x1, f_x2 = f(x1), f(x2)
    df_x1, df_x2 = df(x1), df(x2)

    # ============================================================
    # Шаги 5-7: итерации метода кубической аппроксимации
    # ============================================================
    _x, f_hat, df_hat = x1, f_x1, df_x1
    iterations = 0
    cond1 = cond2 = False
```

```
    MAX_ITER = 200

    while not (cond1 and cond2) and iterations < MAX_ITER:
        iterations += 1

        denom_x = x2 - x1
        if abs(denom_x) < 1e-15:
            # Вырожденный интервал — дальше идти некуда
            break

        # Шаг 5: точка минимума кубического полинома, формулы (9)-
(12)
        z = 3 * (f_x1 - f_x2) / denom_x + df_x1 + df_x2          #
(11)
        disc = z * z - df_x1 * df_x2
        if disc < 0:
            disc = 0.0
        w = np.sqrt(disc) if denom_x > 0 else -np.sqrt(disc)
# (12)

        denom = df_x2 - df_x1 + 2 * w
        mu = 0.5 if abs(denom) < 1e-15 else (df_x2 + w - z) /
denom   # (10)

        if mu < 0:
            _x = x2
# (9)
        elif mu > 1:
            _x = x1
        else:
            _x = x2 - mu * denom_x

        f_hat = f(_x)
        df_hat = df(_x)

        # Шаг 6: условие убывания f(x¯) < f(x1)
        while f_hat >= f_x1 and abs(_x - x1) > 1e-14:
            _x = _x - 0.5 * (_x - x1)
            f_hat = f(_x)
            df_hat = df(_x)

        # Шаг 7: условия окончания
        cond1 = abs(df_hat) <= eps1
        cond2 = abs((_x - x1) / _x) <= eps2 if _x != 0 else True

        # ---- Логирование: все поля заполнены ----
        table['Итерации'].append(iterations)
        table['x1'].append(x1)
        table['x2'].append(x2)
        table['f(x1)'].append(f_x1)
        table['f(x2)'].append(f_x2)
        table['df(x1)'].append(df_x1)
        table['df(x2)'].append(df_x2)
```

```
        table['z'].append(z)
        table['w'].append(w)
        table['mu'].append(mu)
        table['x_hat'].append(_x)
        table['f(x_hat)'].append(f_hat)
        table['df(x_hat)'].append(df_hat)
        table['Условие 1'].append(cond1)
        table['Условие 2'].append(cond2)

        # Шаг 7б: обновление интервала по знаку f'(x¯)
        if not (cond1 and cond2):
            if df_hat < 0:
                x1, f_x1, df_x1 = _x, f_hat, df_hat
            elif df_hat > 0:
                x2, f_x2, df_x2 = _x, f_hat, df_hat
            else:
                break

    printTable("Кубическая аппроксимация", table)

    print(f"\n\nРезультат вычисления по методу КУБИЧЕСКОЙ
АППРОКСИМАЦИИ:")
    print(f"• Точка минимума: x_min = {_x:.6f} ± {eps1}")
    print(f"• Значение в точке минимума: f(x_min) = {f_hat:.6f} ±
{eps2}")
    print(f"• Заняло итераций: {iterations}")
    print("=" * 58 + "\n\n")

    if GRAPHICS_ON:
        viewPlot(flag=True, x_min=_x, f_min=f_hat, start=A, end=B,
title='Кубическая аппроксимация')

    return (_x, f_hat)

GRAPHICS_ON = True
EPS1, EPS2  = 0.0003, 0.003
DX          = 1
A, B        = -10, 6

def mainloop():
    global A, B, DX, EPS1, EPS2, GRAPHICS_ON
    while True:
        print("\n\n======== МЕНЮ ========")
        print("1. Метод квадратичной аппроксимации")
        print("2. Метод кубической аппроксимации")
        print(f"3. Показать график функции на отрезке [{A}; {B}]")
        print("4. Изменить настройки программы")
        print("5. Выход")
        c = int(input("-> "))

        if c == 1:
            quadraticApproximation(A, EPS1, EPS2, DX)
```

```
        elif c == 2:
            cubicApproximation(A, EPS1, EPS2, DX)

        elif c == 3:
            viewPlot(flag=False, start=A, end=B)

        elif c == 4:
            print("\n\n======== НАСТРОЙКИ ========")
            print(f"1. Изменить границы интервала (сейчас: [{A},
{B}])")
            print(f"2. Изменить погрешности (сейчас eps1={EPS1},
eps2={EPS2})")
            print(f"3. Изменить шаг (сейчас dx={DX})")
            print(f"4. {"Выключить" if GRAPHICS_ON else
"Включить"} отображение графиков")
            print("5. В меню")
            s = int(input("-> "))
            if s == 1:
                A, B = map(float, input("Введите значения A и B
через пробел -> ").split())
            elif s == 2:
                EPS1, EPS2 = map(float, input("Введите значения
eps1 и eps2 через пробел -> ").split())
            elif s == 3:
                DX = float(input("Введите новый шаг dx -> "))
            elif s == 4:
                GRAPHICS_ON = not GRAPHICS_ON
            else:
                break

        elif c == 5:
            return

        else:
            print("\n\nНеизвестная команда\n\n")

if __name__ == "__main__":
    mainloop()
```